Phase 2



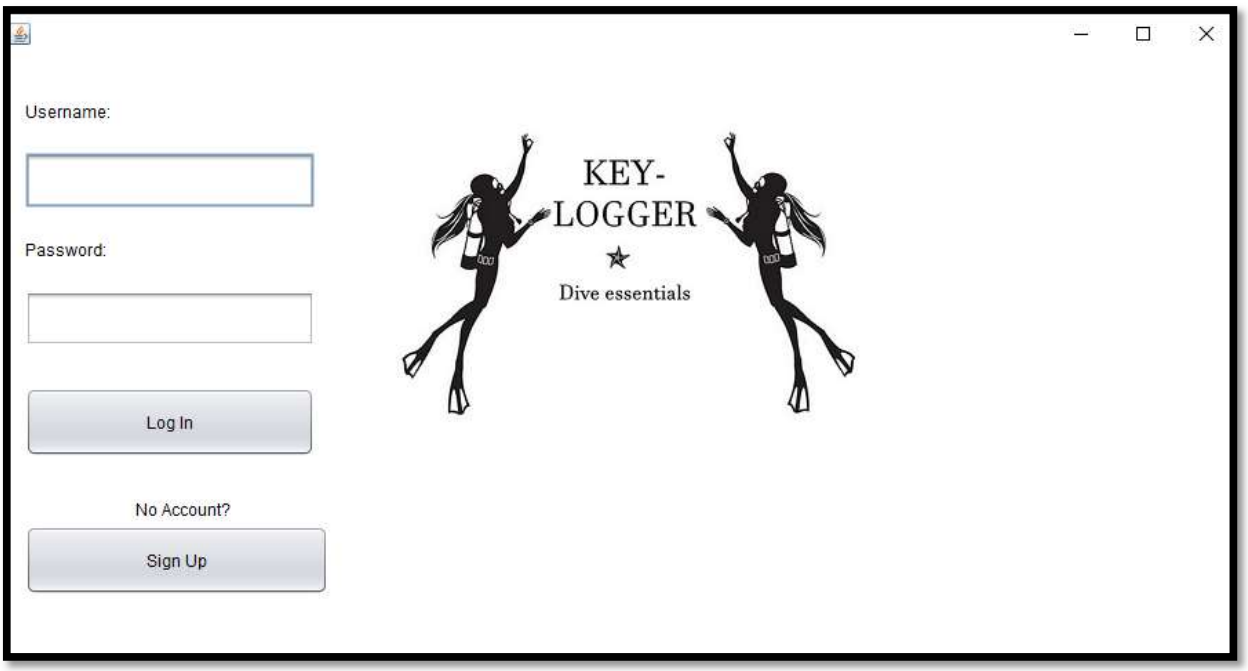
KEY-LOGGER

Dive essentials



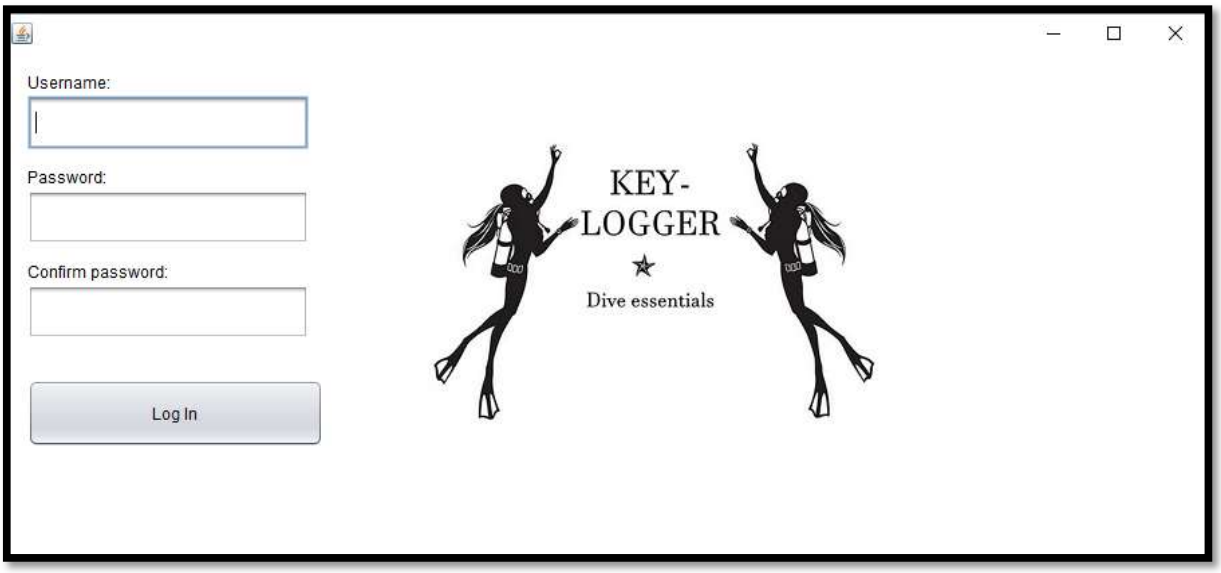
Ruth-ann Rudolph

User Interface design



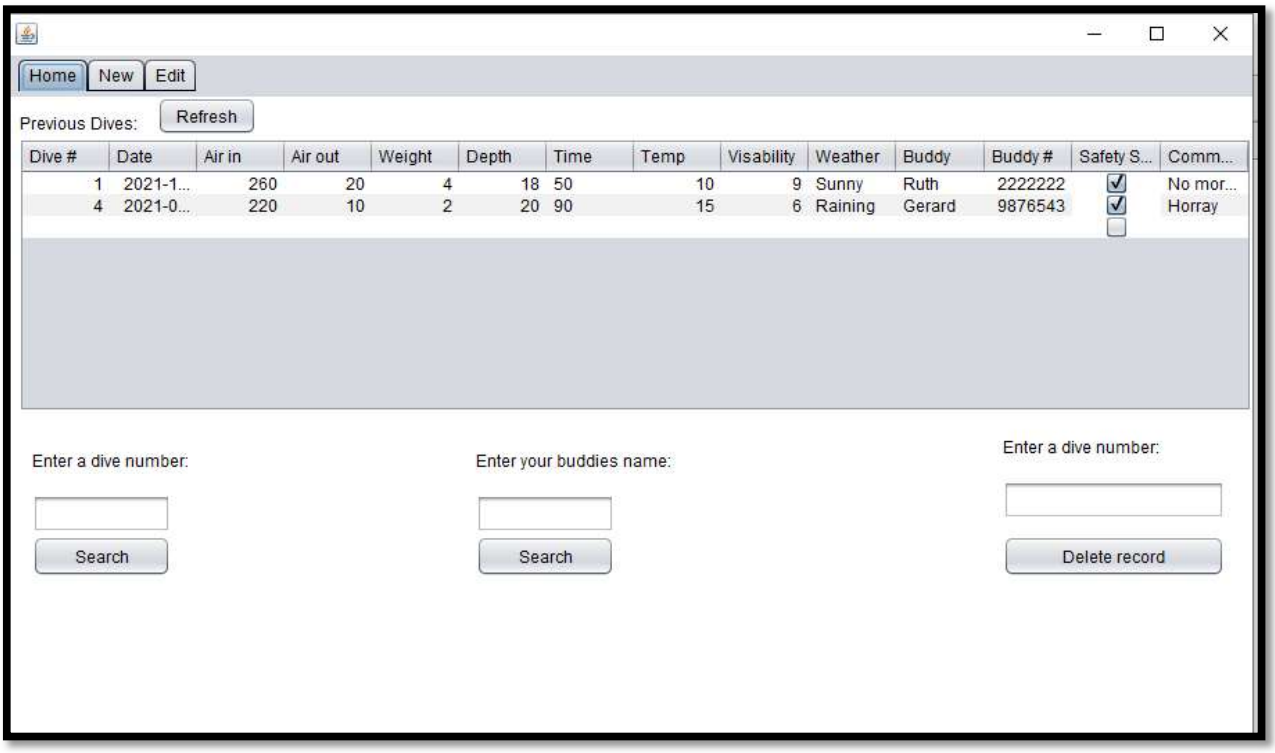
Component	Type
pnlLogIn	JPanel
lblLogo1	JLable
lblUsername1	JLable
lblPassword1	JLable
lblAccount	JLable
btnLogIn	JButton
btnSignUp	JButton
txfUsername1	JTextField
txfPassword1	JTextField

Action	Processing	Output
btnLogIn1 selected	Check that both fields (txfUsername1 and txfPassword1) are filled in. The username and password entered are checked against the existing accounts.	If a match is found, the user may proceed to the home screen, if not, the error screen will be displayed to indicate that invalid information has been entered.
btnSignUp1 selected	pnlLogIn1 is closed	PnlSignUp is displayed



Component	Type
pnlSignUp	JPanel
lblConfirmPassword	JLable
lblLogo	JLable
lblUsername	JLable
lblPassword	JLable
btnSignUp	JButton
txfUsername	JTextField
txfPassword	JTextField
txfConfirmPassoword	JTextField

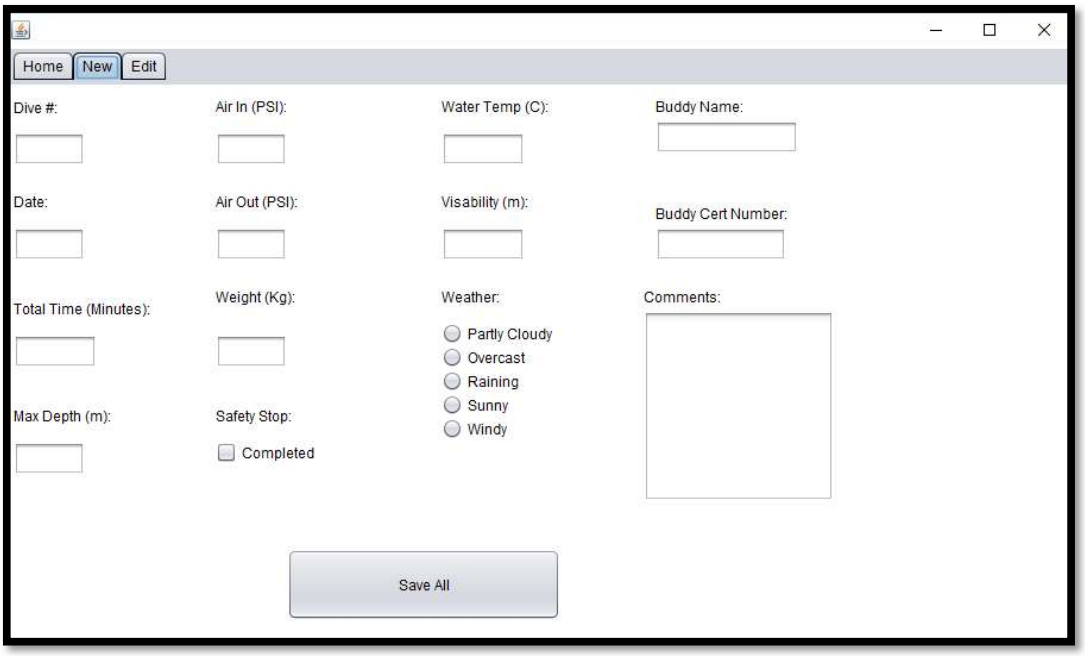
Action	Processing	Output
btnSignUp selected	All 3 fields are checked to be filled in. txfPassword and txfConfirmPassword are checked to be the same.	The user is signed in provided all criteria are met otherwise an error screen will appear to either indicate that all the fields are not filled in or that the passwords do not match.



Component	Type
tbpMain	JTabbedPane
pnlHome	JPanel
lblPreviousDives	JLable
lblDiveSearch	JLable
lblBuddySearch	JLable
lblDiveDelete	JLable
tblLogs	JTable
txfDiveSearch	JTextField
txfBuddySearch	JTextField
txfNumDelete	JTextField
btnNumSearch	JButton
btnBuddySearch	JButton
btnNumDelete	JButton

Action	Processing	Output
btnNumSearch selected	txfDiveSearch is checked to be filled out, the table is searched for that number.	If a match is found it is displayed in the table, if not a message is displayed to indicate that the record was not found.

btnBuddySearch selected	txfBuddySearch is checked to be filled out, the table is searched for that buddy.	If a match is found it is displayed in the table, if not a message is displayed to indicate that the record was not found.
btnNumDelete selected	txfNumDelete is checked to be filled out, the table is searched for that number.	If a match is found it is deleted from the table, if not a message is displayed to indicate that the record was not found.



Component	Type
tbpMain	JTabbedPane
pnlNew	JPanel
lblDiveNum	JLable
lblDate	JLable
lblTime	JLable
lblDepth	JLable
lblAirIn	JLable
lblAirOut	JLable
lblWeight	JLable
lblSafetyStop	JLable
lblTemp	JLable
lblWeather	JLable
lblVisability	JLable
lblBuddy	JLable

lblBuddyNum	JLable
lblComments	JLable
txfDiveNum	JTextField
txfDate	JTextField
txfTime	JTextField
txfDepth	JTextField
txfAirIn	JTextField
txfAirOut	JTextField
txfWeight	JTextField
txfTemp	JTextField
txfVisability	JTextField
txfBuddy	JTextField
txfBuddyNum	JTextField
txfComments	JTextField
cbSafety	JCheckBox
rbgWeatherNew	JRadioButtonGroup
rbnPartlyCloudy	JRadioButton
rbnCloudy	JRadioButton
rbnSunny	JRadioButton
rbnWindy	JRadioButton
rbnRainy	JRadioButton
btnSaveAll	JButton

Action	Processing	Output
btnSaveAll selected	All text fields and other components are checked to be completed.	If all fields are completed a new record is created and saved to the database, if not an error message will appear to indicate that all the fields have not been filled in.

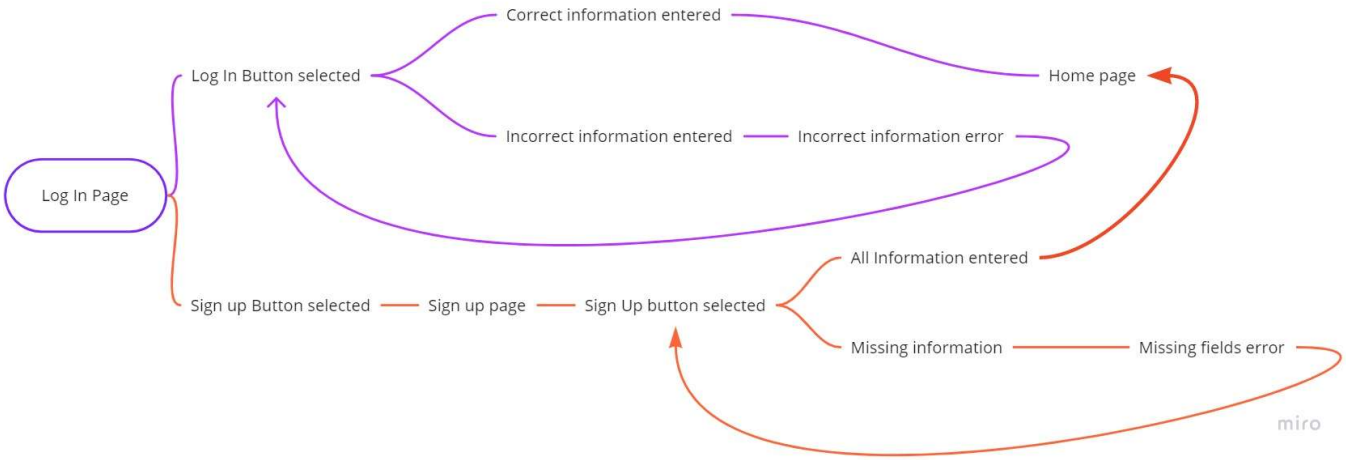


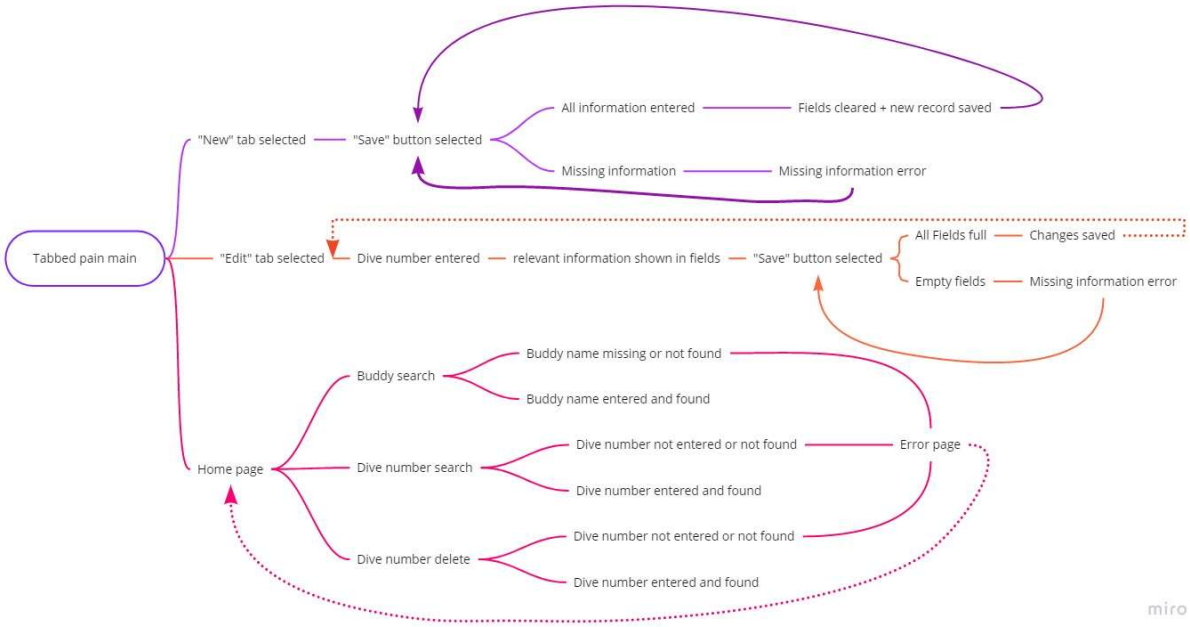
Component	Type
tbpMain	JTabbedPane
pnlEdit	JPanel
lblDiveNumEdit	JLable
lblDateEdit	JLable
lblTimeEdit	JLable
lblDepthEdit	JLable
lblAirInEdit	JLable
lblAirOutEdit	JLable
lblWeightEdit	JLable
lblSafetyStopEdit	JLable
lblTempEdit	JLable
lblWeatherEdit	JLable
lblVisabilityEdit	JLable
lblBuddyEdit	JLable
lblBuddyNumEdit	JLable
lblCommentsEdit	JLable
txfDiveNumEdit	JTextField
txfDateEdit	JTextField
txfTimeEdit	JTextField
txfDepthEdit	JTextField
txfAirInEdit	JTextField
txfAirOutEdit	JTextField
txfWeightEdit	JTextField
txfTempEdit	JTextField
txfVisabilityEdit	JTextField
txfBuddyEdit	JTextField
txfBuddyNumEdit	JTextField

txfCommentsEdit	JTextField
cbSafetyEdit	JCheckBox
rbgWeatherNewEdit	JRadioButtonGroup
rbnPartlyCloudyEdit	JRadioButton
rbnCloudyEdit	JRadioButton
rbnSunnyEdit	JRadioButton
rbnWindyEdit	JRadioButton
rbnRainy Edit	JRadioButton
btnSaveAllEdit	JButton
btnSearchEdit	JButton

Action	Processing	Output
btnSaveAllEdit selected	All text fields and other components are checked to be completed.	If all fields are completed a new record is created and saved to the database, if not an error message will appear to indicate that all the fields have not been filled in.
btnSearchEdit	txfDiveNum is checked to be filled out, the table is searched for that number.	If a match is found all relevant details will be shown in the corresponding fields so that they are available for editing, if not a message is displayed to indicate that the record was not found.

Sequencing


Log In Page
Log In Button selected
Correct information entered
Home page
Incorrect information entered
Incorrect information error
Sign up Button selected
Sign up page
Sign Up button selected
All Information entered
Missing information
Missing fields error
miro


Tabbed pain main
"New" tab selected
"Save" button selected
All information entered
Fields cleared + new record saved
Missing information
Missing information error
"Edit" tab selected
Dive number entered
relevant information shown in fields
"Save" button selected
All Fields full
Changes saved
Empty fields
Missing information error
Home page
Buddy search
Buddy name missing or not found
Buddy name entered and found
Dive number search
Dive number not entered or not found
Error page
Dive number entered and found
Dive number delete
Dive number not entered or not found
Dive number entered and found
miro

Class diagrams

DBLog	Description
- url: String - connect: Connection	url – connects java and access through ucanaccess Connection – connects the database and java
+ DBLog () + queryDatabase (sql: String) : ResultSet + updateQuery (sql: String)	DBLog - Establishes a connection to the database or throws an error queryDatabase - Sends and receives SELECT queries between DB and Java updateQuery - Updates the DB using queries in Java

Log	Description
- diveNum: Integer - diveDate: LocalDate - airIn: Integer - airOut: Integer - weight: Integer - depth: Integer - time: Integer - temperature: Integer - visibility: Integer - weather: String - buddyName: String - buddyNum: Integer - safety: Boolean - comments: String	diveNum - Primary key uniquely identifying the dive diveDate - The date on which the dive took place airIn - The amount of air in the tank at the start of the dive airOut - The amount of air in the tank at the end on the dive weight - The weight used by the diver in kgs depth - The maximum depth reached on the dive time - The time spent under during the dive temperature - The water temperature at the time of the dive visibility - The distance that the diver can see in the water weather - The weather at the time of the dive buddyName - The name of the person partnered with the diver on the dive buddyNum - The dive number of the dive partner on the dive safety - Whether the diver made the recommended safety stop on the dive comments - Any additional information the diver wishes to make note of
+ Constructor (n: Integer, dd : LocalDate, t : Integer, d : Integer, ai : Integer, ao : Integer, w : Integer, s : Boolean, temp : Integer, v : Integer, we : String, bn :	• creates an object with all relevant information from the table

String, bnum : Integer, com : String) + setDiveNum (s: Integer) + setDate (s: LocalDate) + setTime (s: Integer) + setDepth (s: Integer) + setAirIn (s: Integer) + setAirOut (s: Integer) + setWeight (s: Integer) + setSafety (s: Boolean) + setTemp (s: Integer) + setVis (s: Integer) + setWeather (s: String) + setBName (s: String) + setBNum (s: Integer) + setComments (s: String) + getDiveNum ( ) : Integer + getDate ( ) : LocalDate + getTime ( ) : Integer + getDepth ( ) : Integer + getAirIn ( ) : Integer + getAirOut ( ) : Integer + getWeight ( ) : Integer + getSafety ( ) : Boolean + getTemp ( ) : Integer + getVis ( ) : Integer + getWeather ( ) : String + getBName ( ) : String + getBNum ( ) : Integer + getComments ( ) : String + toString () : String	• Set – sets the value to one entered by the user • Get – retrieves the value from the database and returns it • toString – retrieves all the values of the object and returns them as a string

LogManager	**Description**
- lArr: Log - size: Integer - d: DB - modle: DefaultTableModle	• Log object • Database object
+ LogManager () + ShiftLeft(pos: Integer) + insertLog (f : Log) + deleteLog(row: Integer) : String + toString () : String	• Creates a log object for every record in the table • Moves elements of the array to the left • Inserts a new log into the database • Deletes Log from the database

	• toString – retrieves all the values of the object and returns them as a string

LogsUI	**Description**
- dman :DBLogs - model: DefaultTableModel	• Creates database link • Sets a default table model
+ LogsUI( ) + LoadData( ) + btnSignUpActionPerformed( ) + btnLogIn1ActionPerformed( ) + btnSearchEditActionPerformed( ) + btnSignUpActionPerformed( ) + btnSaveAllActionPerformed( ) + btnDiveSearchActionPerformed( ) + btnBuddySearchActionPerformed() + btnDiveDeleteActionPerformed( ) + btnRefreshActionPerformed( ) + btnSaveAll1ActionPerformed( )	• LoadData: Loads data from database into the Jtable • btnSignUp: Changes between the log in and sign-up panels to allow new users to create an account • btnLogIn: checks username and password against current line of the text file • btnSearchEdit: receives a Dive number, finds the dive number, and enters each piece of information into the corresponding field • btnSignUp: Creates a new log in • btnSaveAll: Retrieves information from all fields and creates a new entry in the database • btnDiveSearch: Searches for an entry by dive number and returns it to the table if found • btnBuddySearch: Searches for an entry by buddy name and returns it to the table if found • btnDiveDelete: Retrieves a dive number, searches for it in the database and deletes it if found • btnRefresh: calls the LoadData method • btnSaveAll1: Retrieves information from all fields and updates the entry corresponding with the dive number

Persistent storage design

Field Name	Data Type	Description (Optional)
DiveNumber	Number	Primary key uniquly identifying the dive
DiveDate	Date/Time	the date on which the dive took place
AirIn	Number	The anount of air in the tank at the start of the dive
AirOut	Number	The amount of air in the tank at the end on the dive
Weight	Number	The wieght used by the diver in kgs
Depth	Number	The maximum depth reached on the dive
Time	Number	The time spent under during the dive
Temp	Number	The water temperature at the time of the dive
Visability	Number	The distance that the diver can see in the water
Weather	Short Text	The weather at the time of the dive
NameOfBuddy	Short Text	The name of the person partnered with the diver on the dive
BuddyCertNumber	Large Number	The dive number of the dive partner on the dive
SafetyStop	Yes/No	Weather or not the diver made the reccomended safety stop on the dive
Comments	Long Text	Any aditional information the diver wishes to make note of

Explanation of storage design

Database	
Pros	**Cons**
▪ All items are sorted and organized into the correct categories and all related information is stored together ▪ Data can be sorted, stored and edited directly or through java programs ▪ Data can be securely stored	▪ Requires knowledge and experience in both SQL and java ▪ Requires an external library to access data in java (ucanaccess)
Text Files	
Pros	**Cons**
▪ Small storage requirements ▪ Easy to use for small applications ▪ Integrates easily with java	▪ Poor support for advanced data types ▪ Poor security ▪ Cannot link multiple files